CRAB: A NOVEL CLUSTERING SCORE USING CLUSTERING WITH RIVALS AND BUDDIES FOR UNSUPERVISED LEARNING

A Thesis

presented to

the Faculty of California Polytechnic State University,

San Luis Obispo

In Partial Fulfillment

of the Requirements for the Degree

Master of Science in Statistics

by

Allen Choi

June 2026

© 2026

Allen Choi

ALL RIGHTS RESERVED

COMMITTEE MEMBERSHIP

TITLE:	CRAB: A Novel Clustering Score using Clustering with Rivals and Buddies for Unsupervised Learning
AUTHOR:	Allen Choi
DATE SUBMITTED:	June 2026

COMMITTEE CHAIR:	Kelly Bodwin, Ph.D. Thesis Advisor
COMMITTEE MEMBER:	Alexander Dekhtyar, Ph.D. Professor of Computer Science and Software Engineering
COMMITTEE MEMBER:	Kevin Ross, Ph.D. Professor of Statistics

ABSTRACT

CRAB: A Novel Clustering Score using Clustering with Rivals and Buddies for Unsupervised Learning

Allen Choi

Unsupervised clustering algorithms today are used across a wide variety of fields such biology, engineering, and industry in order to classify observations into groups where labels are not provided. This can provide important latent information regarding the observations within groups, as well as insight regarding the groups themselves. In order to judge the optimal number of clusters for an unsupervised clustering algorithm, many methods exist such as the elbow method and Silhouette Score; however, these methods come with drawbacks and are not necessarily flexible across many unsupervised methods. We present a novel clustering score framework relying on a resampling-based approach that classifies pairs of points as either ”rivals” or ”buddies”, constructing a score for a given number of clusters. We then present results demonstrating the properties of this metric on various simulated and real datasets, measuring the performance of the algorithm as well.

Keywords: Clustering. Unsupervised Clustering. Machine Learning.

ACKNOWLEDGMENTS

I'd like to first thank my amazing Mom and Dad for providing me so much emotional support over these past half a decade at Cal Poly. Without them, I don't know how I would have gotten through college.

But without a doubt, in regards to my statistical knowledge and skills I have no one else to thank but the amazing Statistics faculty at Cal Poly, who have always been patient, understanding, and kind.

Also a thank you to Andrew Kerr for providing a quarto template: https://github.com/Wander03/CalPolySTAT-thesis-template.

TABLE OF CONTENTS

Page

LIST OF TABLES vii

LIST OF FIGURES viii

CHAPTER

1 Introduction 1

1.1 Motivation 1

1.1.1 Elbow Method 2

1.1.2 Silhouette Score 4

1.2 Overview of k-Means Clustering 5

1.2.1 k-Means Clustering Details 6

1.3 Literature Review 6

2 Algorithm Details 9

2.1 CRAB Score Algorithm 11

2.2 Known Data 12

2.2.1 S-sets Datasets 12

2.2.2 Random Normal Data 15

2.3 Performance on penguins dataset 17

3 Computational Details 20

3.1 Matrix Stacking 20

3.2 Algorithm Construction 21

3.3 Score Construction 23

3.4 Speed and Memory Scaling 27

3.4.1 Scaling with the Number of Observations 27

3.4.2 Scaling with the Number of Dimensions 29

4 Null Distribution 32

4.1 Null Distribution of Scores 32
4.2 Null Distribution over Change in Number of Dimensions 34
4.3 Null Distribution over Change in Number of Observations 35
4.4 Adj-CRAB Score 36
5 Conclusion 38
5.1 Results 38
5.2 Limitations and Future Work 38
REFERENCES 39
APPENDICES
A Resample Matrix 42
B Mean Matrix 45
C Metric Calculation 46
D Point Contribution Function 47
E Get Point Vector Helper 48
F CRAB Score Across k Clusters 49

LIST OF TABLES

Table Page

2.1 Distribution of Adj-CRAB Scores over k-Clusters (30 Simulations per Cluster) 18

LIST OF FIGURES

Figure Page

1.1 Scatterplot of the Example Dataset 3

1.2 WCSS Decreases as Number of Clusters k Increases 3

1.3 Silhouette Score as Number of Clusters k Increases 5

1.4 Three 60% Resamples of the iris dataset for Petal Length vs. Petal Width . . 7

1.5 Three 60% Resamples of a 2-D Random Normal Distribution 8

2.1 Pairs of points considered 'buddies' within a specific resample 10

2.2 Pairs of points considered 'enemies' within a specific resample 10

2.3 Visualizations of the S-sets Datasets (S1, S2, S3, S4) after scaling 13

2.4 Distribution of adj-CRAB scores across k = 2 through k = 30. The true number of clusters is 15. 14

2.5 Zoomed-In distribution of adj-CRAB scores across k = 8 through k = 20. The true number of clusters is 15 15

2.6 Four random simulated Gaussian clusters (Scaled) 16

2.7 CRAB Score on simulated dataset across k = 2 to k = 10 17

2.8 Distribution of Adj-CRAB Scores over k-Clusters (30 Simulations with different starting seeds) 18

2.9 a 19

3.1 Example dataset from Section 2.1 with cluster assignments removed (2 Distributions) 21

3.2 CRAB algorithm ran on Section 2.1 Dataset with k = 2 clusters 22

3.3 CRAB algorithm ran on Section 2.1 Dataset with k = 3 clusters 23

3.4 Plot of Penguin Species by Standardized Bill and Flipper Length 24

3.5 The distribution of values inside the mean matrix after running the algorithm on the penguins dataset, using K-Means and k = 2 25
3.6 The distribution of values inside the mean matrix after running the algorithm on the penguins dataset, using K-Means and k = 3 25
3.7 Speed of the CRAB Algorithm Quadratically Increases as Number of Observations Increases 28
3.8 Memory of the CRAB Algorithm Quadratically Increases as Number of Observations Increases 29
3.9 Speed of the CRAB Algorithm Increasing Linearly as Number of Dimensions Increases 30
3.10 Memory of the CRAB Algorithm Increasing Linearly as Number of Dimensions Increases 31

4.1 Distribution of Scores on a Random 2-Dimensional Normal Distribution, Faceted by Resample % 33
4.2 Distribution of Scores on a p-Dimensional Normal Distribution, Faceted by Number of Dimensions 34
4.3 Distribution of scores on random p-dimensional normal distribution (proportion resample set to 80%) 35
4.4 Distribution of scores across various resample percentages on random p-dimensional normal distribution (n = 200) 36
4.5 CRAB Score Behavior as k Increases (Fitted Curve vs. Actual Simulation Results) Approximated by a Quartic Regression 37

Chapter 1

INTRODUCTION

1.1 Motivation

Unsupervised clustering algorithms are commonly used today when attempting to classify observations in groups where the observation's membership or the number of groups is not known beforehand. Clustering is useful when attempting to get an overview of groups across a dataset, which provides insight into which data points are most similar to other data points. This can provide implicit signals into common traits shared by observations in the same group, which is valuable across a wide range of fields such as biology (Nugent & Meila, 2010), engineering (Ababneh, Khodier, & Khatalin, 2025), health education (Roper, He, Perez-Concha, & Guthridge, 2023), physiology (Bird et al., 2022), and industry (John, Shobayo, & Ogunleye, 2023).

One of the main difficulties associated with unsupervised clustering is due to the fact that the number of clusters or groupings in the dataset is not known beforehand. This can be circumvented through the use of domain knowledge, but even in situations with no prior knowledge there still exist methods to estimate the optimal number of groups. Methods such as the elbow method and Silhouette score (Kodinariya & Makwana, 2013) are commonly used to compare multiple candidate values of k over a range to obtain a specific "best-guess" of this optimal value. However, even with these methods there are often disagreements and caveats that we will further explore. In this thesis we introduce a novel method for determining the optimal number of clusters using a resampling-based approach over many subsamples of a dataset.

1.1.1 Elbow Method

The elbow method (Thorndike, 1953) is a relatively simple technique that utilizes the plotting of the sum of the Within Cluster Sum of Squares (WCSS) across all clusters at the y axis, against the number of clusters on the x-axis. Adding more clusters necessarily decreases the WCSS due to the fact that adding an extra cluster will mean that at least one observation is closer to that cluster than the others. The question of "how many clusters" is then answered by looking at where this drop-off in WCSS is significant, as seen in the example dataset below. This method however comes with many problems (Schubert, 2023). The biggest issue is that while for some datasets this "drop-off" in WCSS is easy to visually recognize, sometimes it can be extremely difficult to tell what is considered significant. In fact, Schubert takes a strong approach on this, where they state "Educators should omit the method or at least explain better alternatives".

We demonstrate this below with an example dataset, where:

$$x_1, y_1 \sim gamma(\alpha = 1)$$
$$x_2, y_2 \sim gamma(\alpha = 1.5) + 2$$
$$x_3, y_3 \sim gamma(\alpha = 2) + 4$$

Fifty observations were randomly generated from each distribution, for a total of 150 observations (X, Y) across the three distributions.

An overview of this data can be seen below, where the observations are more obviously separated into three clusters, as taken from their respective distributions.

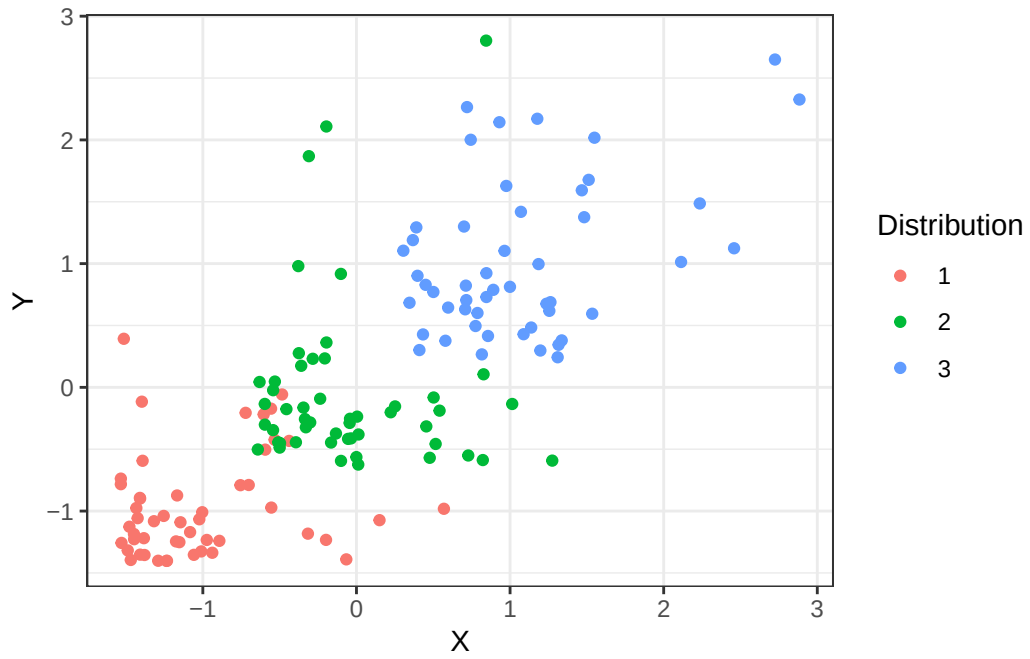

Figure 1.1: Scatterplot of the Example Dataset

Using this data, we used k-Means clustering across a set of candidate k cluster values from one to ten, visualizing the WCSS across all the k values.

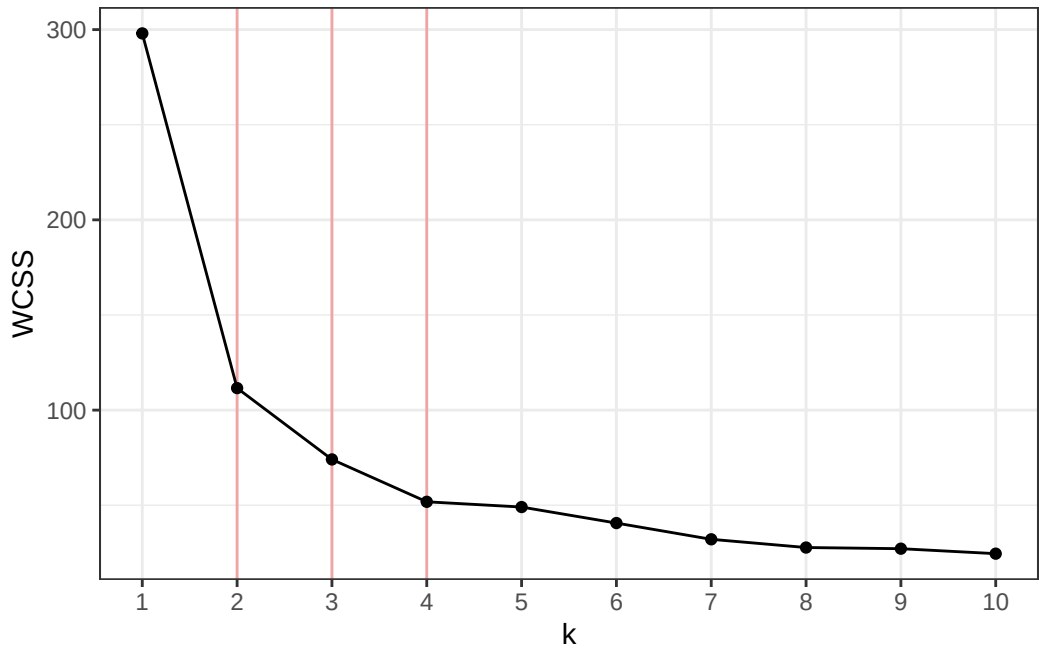

Figure 1.2: WCSS Decreases as Number of Clusters k Increases

As seen in the plot above, there exists no obvious "elbow" at which point the decrease in WCSS changes dramatically; any choice of k between 2 and 4 is a justifiable answer, which makes the ideal choice of k ambiguous.

1.1.2 Silhouette Score

The Silhouette Score approach (Rousseeuw, 1987) is a method proposed to measure how comfortably each observation belongs to its assigned cluster compared to the other possible clusters. As it is defined in the original 1986 paper, the silhouette width $s(i)$ for observation i is defined by:

$$s(i) = \frac{b(i) - a(i)}{max\{a(i), b(i)\}}$$

where $a(i)$ is defined by the average distance between the i and all other observations in the same cluster, and $b(i)$ is defined by the smallest average distance between i and all other observations in each other respective clusters.

Averaging this score gives a measure of how well a certain number of clusters fits the data, similar to the elbow method. Unlike in the elbow method however, where increasing the number of clusters necessarily decreases the WCSS, the average Silhouette width does not necessarily increase as the number of clusters increases. A high Silhouette Score for a given k means that k is a good number of clusters for that given dataset.

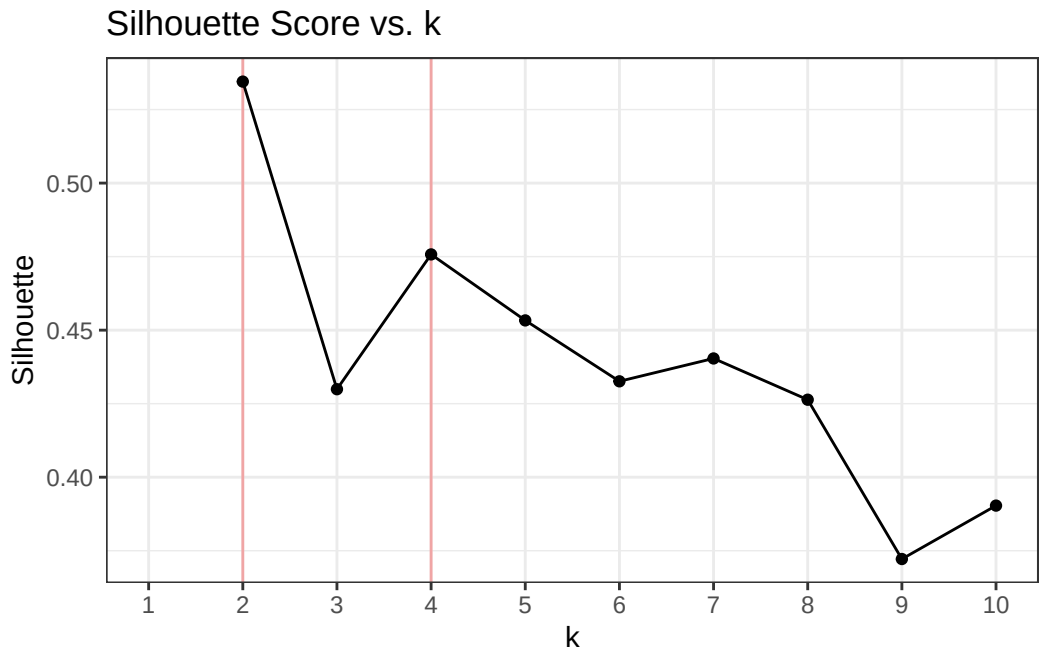

Figure 1.3: Silhouette Score as Number of Clusters k Increases

When looking at the Silhouette Scores on the same example dataset where the elbow method was used, we see that the Silhouette Score are highest for $k = 2$ and next-highest for $k = 4$, indicating that the optimal number of clusters is 2 or 4 for this dataset. Since we know beforehand that the data is distributed from three gamma distributions, this answer is not "correct".

1.2 Overview of k-Means Clustering

For the scope of this paper we will go over only the k-Means clustering algorithm (MacQueen, 1967). However, in theory the CRAB algorithm framework allows it to work on any unsupervised method as long as the algorithm outputs a cluster assignment for every observation in the dataset. Alternatives such as hierarchical agglomerative clustering and mixture models (Jaeger & Banks, 2023) would also work using this framework.

1.2.1 k-Means Clustering Details

In terms of unsupervised clustering, k-Means clustering is one is the most well-known as well as simplest methods available. In essence, the algorithm seeks to find clustering assignments that minimize the average squared distance between a set of points and various centroids k that are randomly initialized in a dataset. The clustering results are highly dependent on the initial starting positions of the centroids due to the fact that k-Means often converges to local minima, and when the usual Euclidean distance metric is used it often finds spherical clusters that are centered around these centroids (Ikotun, Ezugwu, Abualigah, Abuhaija, & Heming, 2023). The speed and memory complexity of the algorithm will be explored in Section 3.5.

1.3 Literature Review

Much previous work has been done in leveraging resamples in order to inform the number of clusters. One paper computes clustering assignments completed on random resamples on the full dataset, then comparing those resampled cluster assignments against the clustering assignment completed on the full dataset (Levine & Domany, 2001). Another paper obtains the clustering assignments on two random subsamples of a dataset, then computes a metric on how similar the assignments are between those two resamples (Ben-Hur, Elisseeff, & Guyon, 2001). We seek to extend this concept by running a large number of resamples on random resamples of a dataset, then combining the information obtained from each respective cluster assignments into a score on a cluster-by-cluster basis. For most experiments shown in the thesis unless explicitly stated, we use the same resampling proportion of 80% as stated in Ben-Hur and et al.'s, paper.

We demonstrate the basic concept of resampling and the stability of cluster assignments using the iris dataset (FISHER, 1936) below.

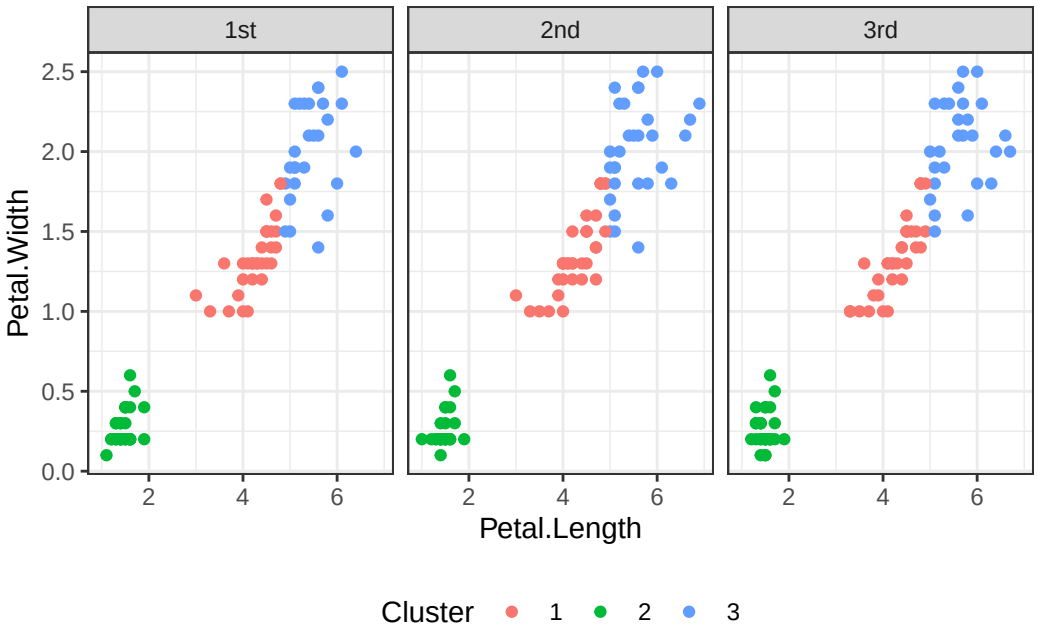

Figure 1.4: Three 60% Resamples of the iris dataset for Petal Length vs. Petal Width

As seen in the above figure, even with random resamples, the general location of the clusters are similar. More importantly, though, is the fact that observations that are in the same cluster tend to stay in the same cluster, given that there is a clear clustering structure observable.

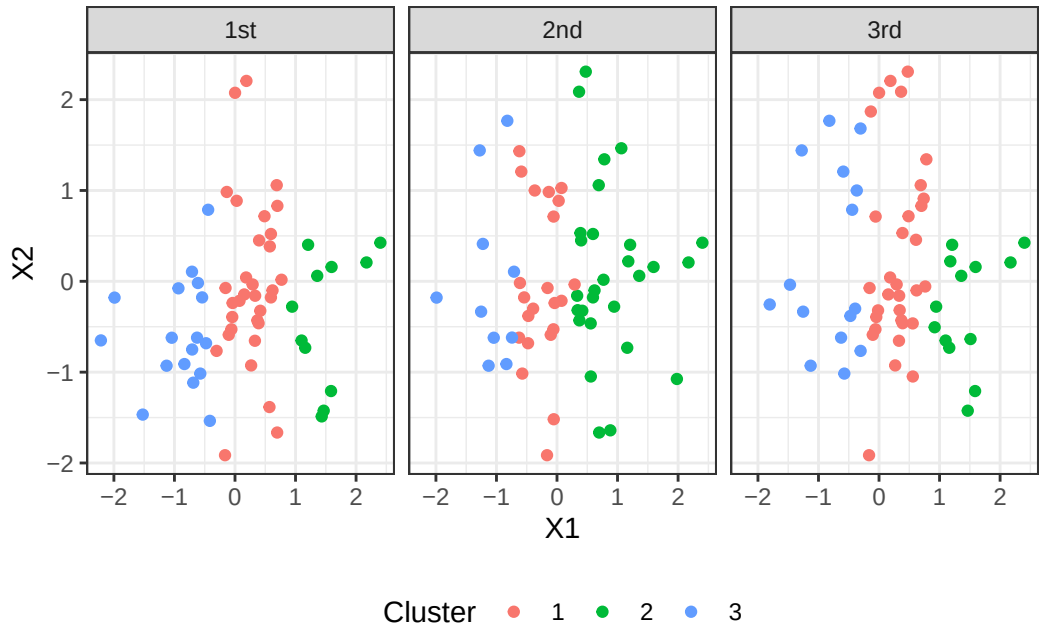

Figure 1.5: Three 60% Resamples of a 2-D Random Normal Distribution

As seen in the plot above, when the clustering structure is not clearly identifiable through k-Means, the clustering assignments outputted from k-Means tends to be significantly different from resample to resample. This is in contrast to the iris dataset example above with three 60% resamples; the clustering assignments tend to be similar even after the resampling procedure was completed.

Chapter 2

ALGORITHM DETAILS

The main mechanism the CRAB algorithm attempts to leverage is the fact that on a given dataset, if there is an obvious clustering assignment present for the observations, then that overall clustering should also be present in a random subsample of the dataset. In this case, barring very unusual and unlucky resamples, the location of the clusters in a dataset should stay consistent across resamples.

Given two distinct points and their cluster assignments across resamples, the clustering assignments should be consistent if there is an obvious clustering pattern in the dataset. For example, two points that are far away from each other should consistently not be in the same cluster, and two points that are close together should consistently be in the same cluster, even across resamples. In k-Means specifically, this means that there should be spherical clusters that are relatively far apart for the cluster assignments to be consistent across resamples; a large overlap between groups means that after even after a resample, a large proportion of observations will be not in the same cluster or "enemies" with their neighboring observations, and vice versa.

We demonstrate this concept with a example dataset below.

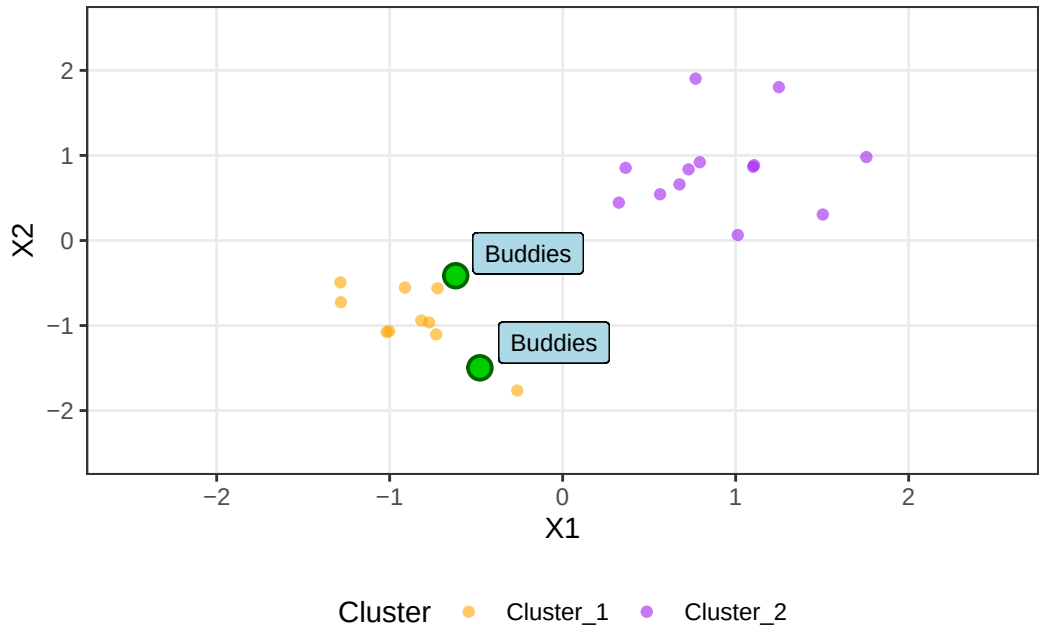

Figure 2.1: Pairs of points considered 'buddies' within a specific resample

Due to the relative closeness of these two points, we see that any given resample should have these two points in the same cluster even across many resamples.

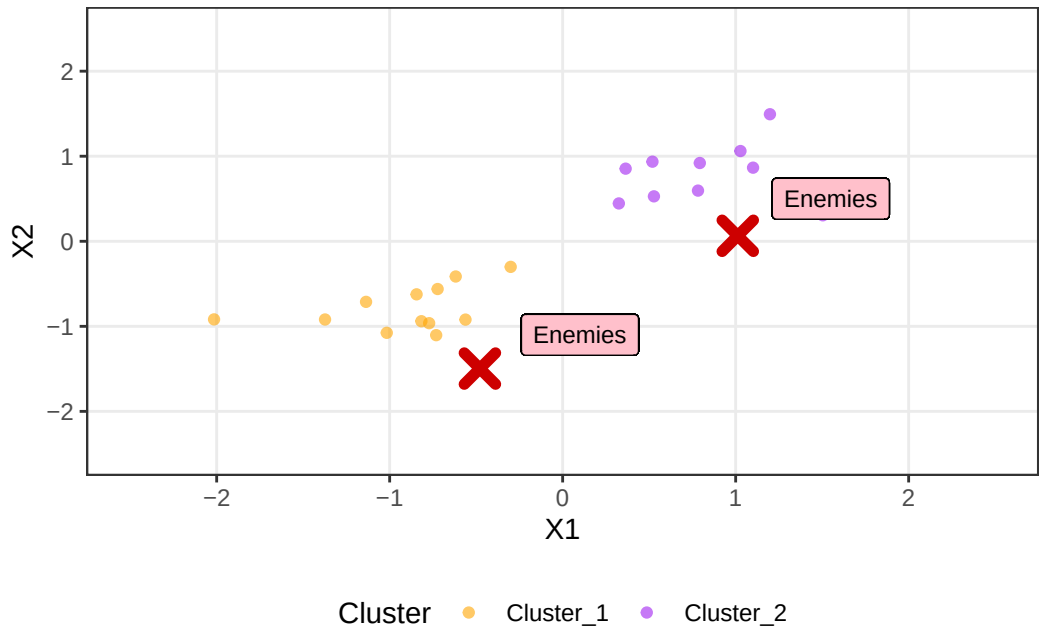

Figure 2.2: Pairs of points considered 'enemies' within a specific resample

Due to the relative far distance of these two points, we saw that any given resample would have these two points in the different cluster even across many resamples. From here, we derived the main idea our algorithm is based off of: across resamples the pairwise relationship between observations should be consistently either "buddies" or "enemies", but not both.

2.1 CRAB Score Algorithm

We give a high-level overview of the CRAB algorithm details and workflow below:

1. Select m random subsamples of the data, each subsample a certain percentage of the full dataset.
2. On all m subsamples of the data, use an unsupervised clustering algorithm to obtain cluster assignments for every observation on a subsample by subsample basis.
3. For each pair of distinct points i, j in the whole dataset, in a given subsample, record whether they were in the same cluster, not in the same cluster, or either point i or point j were in the subsample. Repeat for all m resample matrices.
4. Analyze the consistency of agreements or disagreements of the cluster assignments of the pairwise points across the m subsamples.
5. Repeat the above steps for a set of k candidate values for the number of clusters, passing in a different k into the unsupervised clustering algorithm.

The main parameters the user has control over when utilizing the algorithm are:

- *formula*: The variables to use when running the CRAB algorithm; by default, it utilizes every variable in the dataset.
- *number_of_resamples*: The number of resample matrices to construct when running the CRAB algorithm; by default, it uses 15 resample matrices which may not be enough for large datasets where more resamples are needed to obtain a stable score.
- *proportion_resample*: The proportion of the full dataset that the CRAB algorithm randomly samples; by default it is set to 80%, and behavior exploring changing this

setting is provided in Section 4.

- *algorithm*: The algorithm to use when supplying the cluster assignments to the CRAB algorithm; by default it is set to kmeans, and hierarchical clustering through 'hier_clust' is supported experimentally.

The full function used for the algorithm is available in Appendix A.

2.2 Known Data

We validate the performance of the CRAB algorithm through the use of simulated data as well as other datasets with a known number of clusters to compare against.

2.2.1 S-sets Datasets

There are a significant number of datasets available online in order to gauge the performance of unsupervised clustering algorithms (Pasi Fränti & Sieranoja, 2018). In this thesis we use a specific subset of those datasets called the 'S-sets', which consist of two-dimensional Gaussian data with 5000 observations and 15 clusters with different overlap over the four different S-sets (P. Fränti & Virmajoki, 2006). These datasets have a few properties that are desirable for measuring the performance of the CRAB algorithm: each dataset has a constant number of 5000 observations, with a constant number of 15 clusters, and with Gaussian clusters. The consistent observations and clusters over the datasets allow for more accurate comparisons in performance, while the Gaussian clusters synergize well when evaluating the performance when using k-Means clustering with the CRAB algorithm.

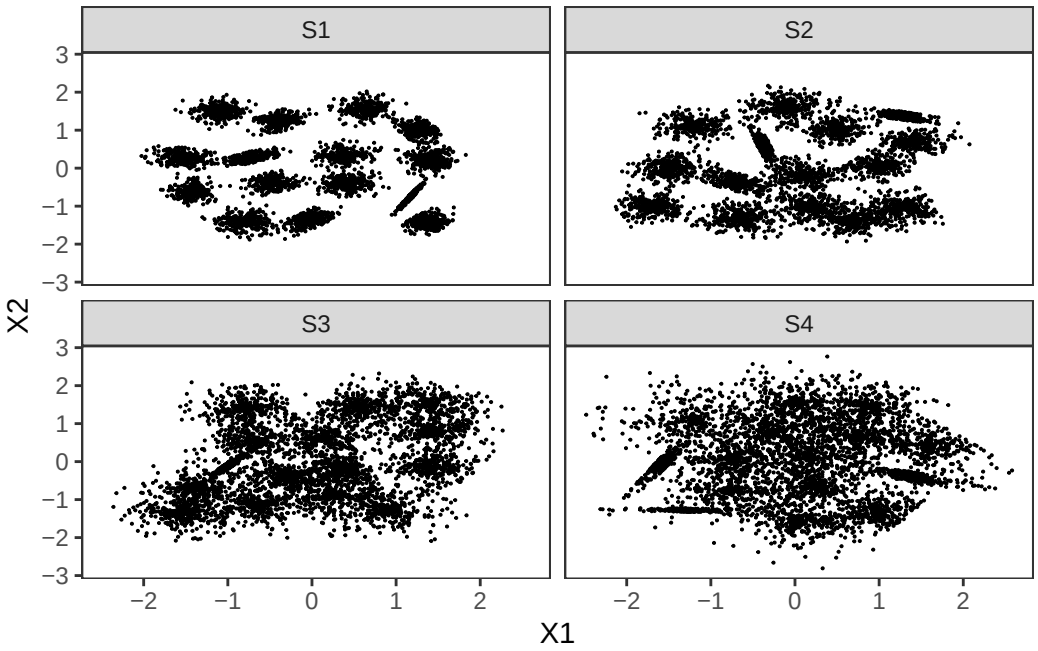

Figure 2.3: Visualizations of the S-sets Datasets (S1, S2, S3, S4) after scaling

We took 50% random points from the dataset due to computation limits. As seen in the above figure, the overlap between the 15 Gaussian clusters becomes more and more extreme as we increment through the datasets.

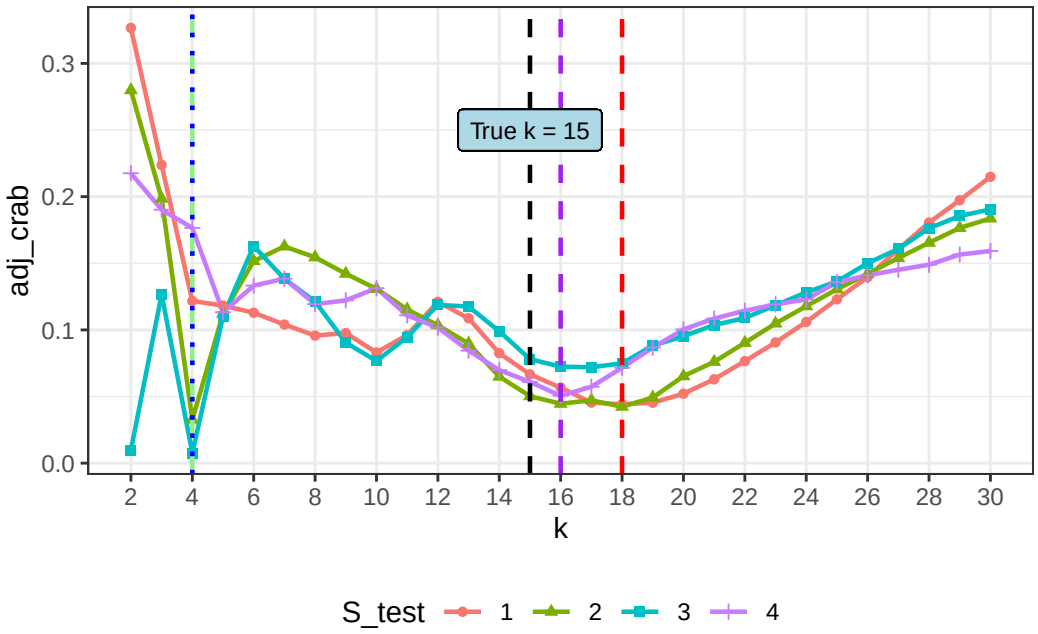

Figure 2.4: Distribution of adj-CRAB scores across k = 2 through k = 30. The true number of clusters is 15.

When running the CRAB algorithm across all these datasets, we see that the CRAB algorithm does not perfectly predict the number of clusters on any of the datasets. However, for datasets S1 and S4 it gets extremely close at 18 and 16 predicted clusters, respectively. For S2 and S3 the minimum CRAB score occurs at k = 4, which is extremely different from the true number of clusters at 15. This seems to suggest that there is something inherent in those datasets specifically because the algorithm run on S1 and S4 does not have identical behavior. Barring this, however, all four trends of adjusted CRAB scores at least appear to have a dip at or near the true number of clusters at 15. It is possible that changing out clustering algorithm or running the algorithm on the full dataset instead of the dataset with 50% of observations will improve performance.

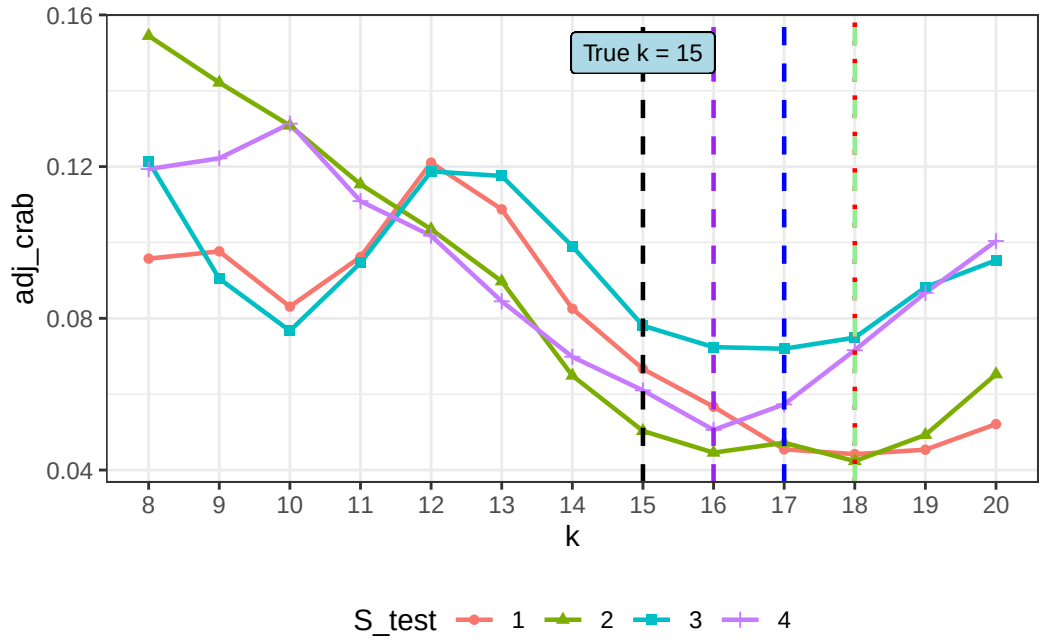

Figure 2.5: Zoomed-In distribution of adj-CRAB scores across k = 8 through k = 20. The true number of clusters is 15

When ignoring the candidate cluster values before 8, the results look significantly more promising. Across all datasets, we get a predicted cluster number of 18, 18, 17, and 16 across the S1, S2, S3, and S4 datasets respectively. Interestingly, the algorithm seems to have consistent performance across the datasets even as the overlap between clusters increases.

2.2.2 Random Normal Data

To illustrate the CRAB algorithm working on smaller simulated random data, we created a dataset of 4 Gaussian clusters where the points are distributed as such:

$$x_1, y_1 \sim Normal(\mu = 1, \sigma = 1)$$

$$x_2, y_2 \sim Normal(\mu = 4, \sigma = 1)$$

$$x_3, y_3 \sim Normal(\mu = 7, \sigma = 1)$$

$$x_4 \sim Normal(\mu = 1, \sigma = 1)$$

$$y_4 \sim Normal(\mu = 7, \sigma = 1)$$

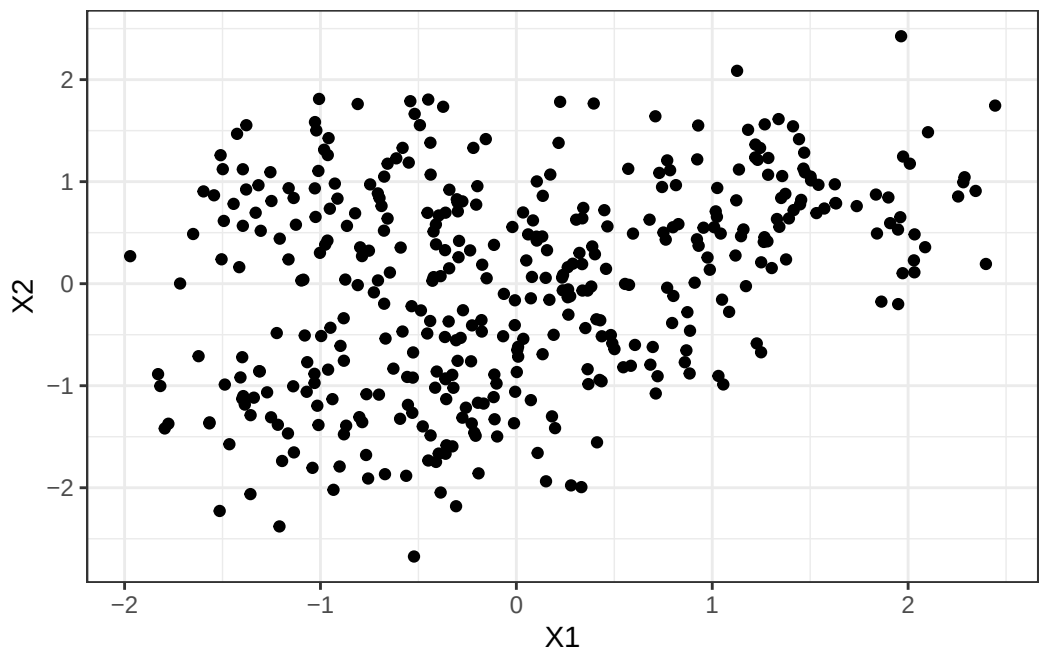

Figure 2.6: Four random simulated Gaussian clusters (Scaled)

The simulated dataset has a significant amount of overlap between the four Gaussian clusters, and it is still relatively possible to pick out the individual groups. Since we simulated four Gaussian clusters, we should expect the CRAB algorithm to output a CRAB score that is lowest for $k = 4$, and CRAB scores higher for $k \neq 4$.

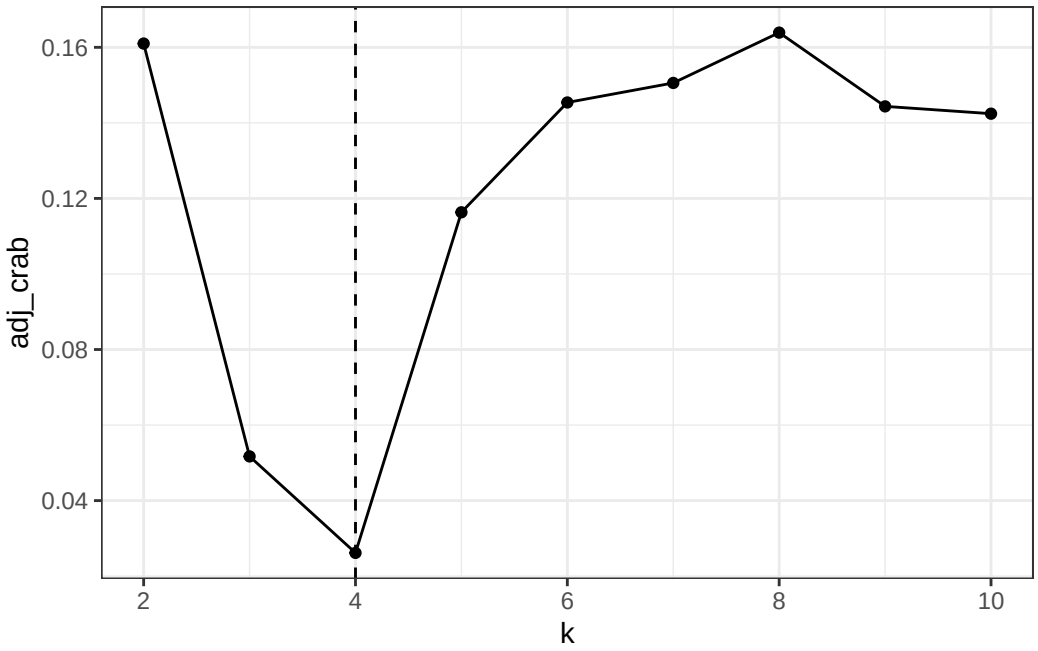

Figure 2.7: CRAB Score on simulated dataset across k = 2 to k = 10

On this dataset we see that the CRAB algorithm works extremely well, correctly identifying the number of clusters in the dataset at 4. Barring $k = 3$, all other options are not close to being chosen as a candidate.

2.3 Performance on penguins dataset

We also wanted to test the performance of the CRAB algorithm on an applied dataset from the real world. The penguins dataset (Horst, Hill, & Gorman, 2022) was chosen for this since it is commonly used and recognizable. The number of species (3) is also known beforehand, providing a convenient way to measure cluster algorithm performance.

We used the **bill_length** variable as well as the **flipper_len** variable from the penguins dataset, standardizing each variable before applying the CRAB algorithm. A graph of the dataset used can be seen in Figure 3.4.

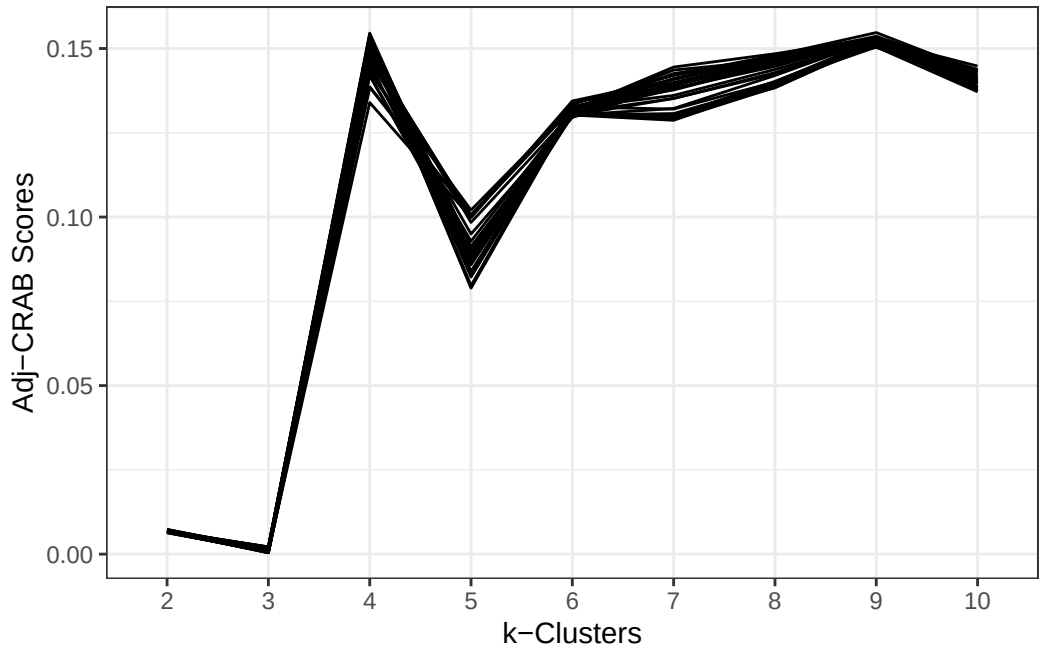

Figure 2.8: Distribution of Adj-CRAB Scores over k-Clusters (30 Simulations with different starting seeds)

To test whether any results we find with the CRAB algorithm is dependent on the starting seed, we ran the CRAB algorithm 30 times on the same penguins dataset, randomly changing the seed each time. We see from the graph that the dip at $k = 3$ and the subsequent increase in the adjusted CRAB score at $k = 4$ is consistent across all starting values of the seed, suggesting that there is little variability between scores within each k, at least with the parameters used for this simulation.

Table 2.1: Distribution of Adj-CRAB Scores over k-Clusters (30 Simulations per Cluster)

k	Average adj_crab
2	0.0068280
3	0.0011021
4	0.1487305
5	0.0902379

k	Average adj_crab
6	0.1316196
7	0.1366056
8	0.1440248
9	0.1522450
10	0.1404753

We see that across the 30 random seeds intializing the CRAB algorithm, the average adjusted CRAB Score is lowest for $k = 3$ compared to the other values of k for the penguins dataset.

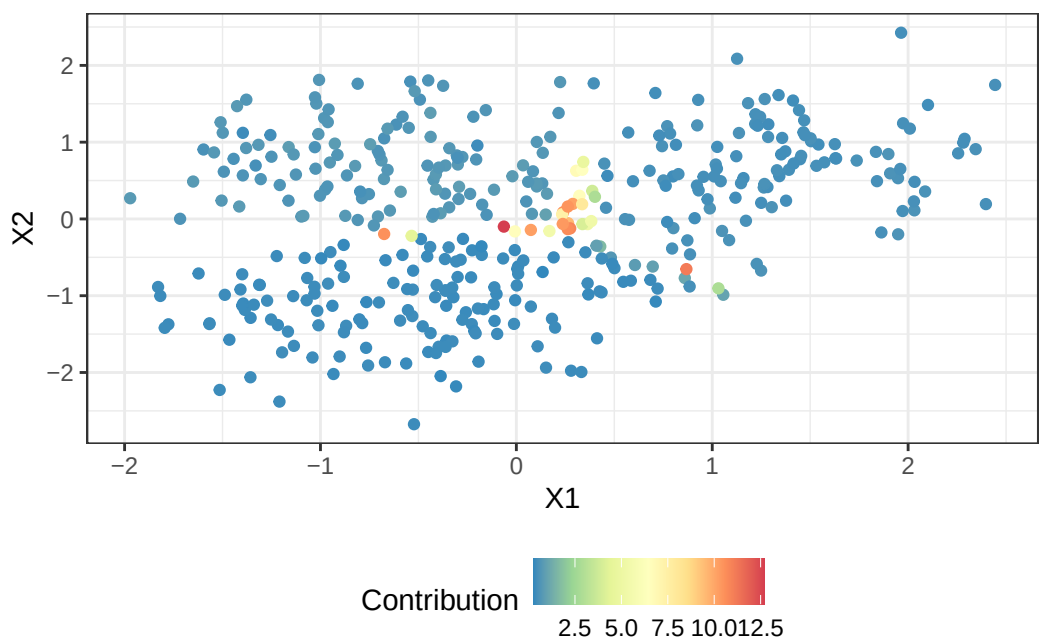

Figure 2.9: a

TODO

Chapter 3

COMPUTATIONAL DETAILS

In this section we go further into detail the implementation of the CRAB algorithm as well as the underlying structures that go into the computation of the CRAB Score.

https://github.com/TheAllenChoi/Cluster_Tuning_Allen

3.1 Matrix Stacking

In Section 2, we described a framework where "buddies" and "enemies" were compared across resamples. To accommodate for this, we needed a way to hold pairwise information on a resample-by-resample basis. A matrix was the obvious solution for this, where row i and column j describes the pairwise relationship between observation index i and observation index j. Since the information is duplicated across the $y = x$ line, where $(i, j) = (j, i)$, we set the entire upper half of the matrix as NA values.

We formally define a "resample matrix" or "cluster assignment matrix" for each resample, where for an $n \times n$ resample matrix m, for every pairwise interaction between observation i and observation j we have a value of 1 in the matrix if they belong to the sample cluster, -1 if not, and 0 if either point was not in the resample.

Denoted another way, for two points i, j in the resample matrix, the element p_{ij} of the resample matrix is defined as:

$$p_{ij} = \begin{cases} 1, & \text{if points i, j belong to the same cluster} \\ -1, & \text{if points i, j do not belong to the same cluster} \\ 0, & \text{if either point i or point j are not in the resample} \\ \text{NA}, & \text{if pairwise element ij is located in the lower triangle or diagonal} \end{cases}$$

3.2 Algorithm Construction

After we have m matrices, we then reduce the matrices, taking the mean with respect to each index pair, ignoring zeroes. We then receive a final "mean matrix" of size n x n, where n is the number of observations in the dataset.

We refer back to the sample dataset we used in Section 2.1. to demonstrate the concept of the 'mean matrix'.

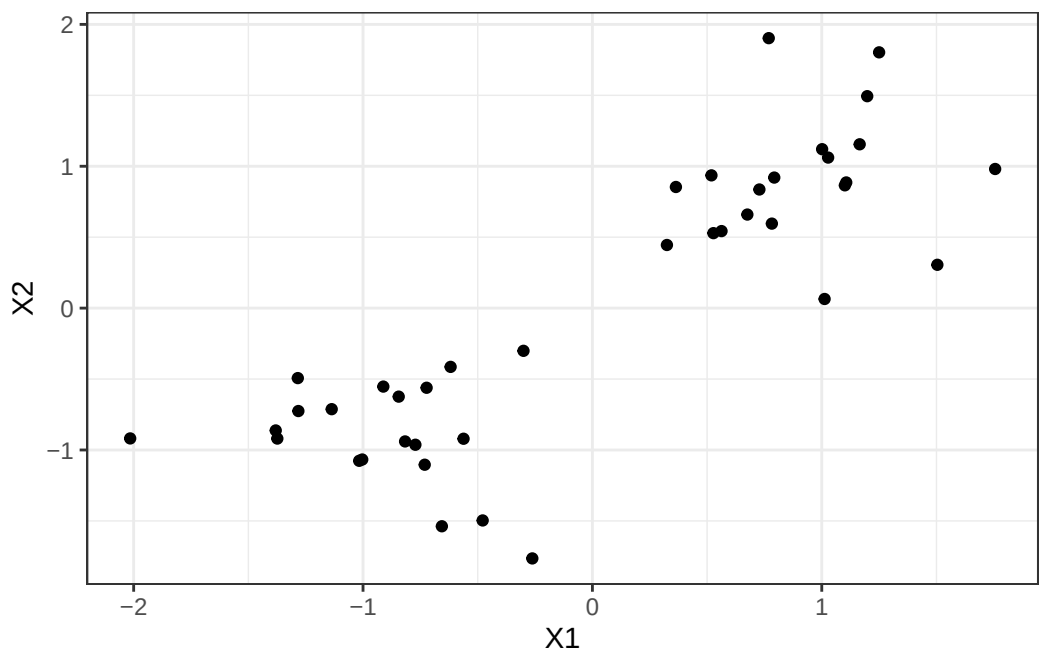

Figure 3.1: Example dataset from Section 2.1 with cluster assignments removed (2 Distributions)

Although the cluster assignments have been removed, we see that there are still distinctly

two clusters in this dataset. The full mean matrix that is calculated after collapsing the resample matrices is shown below.

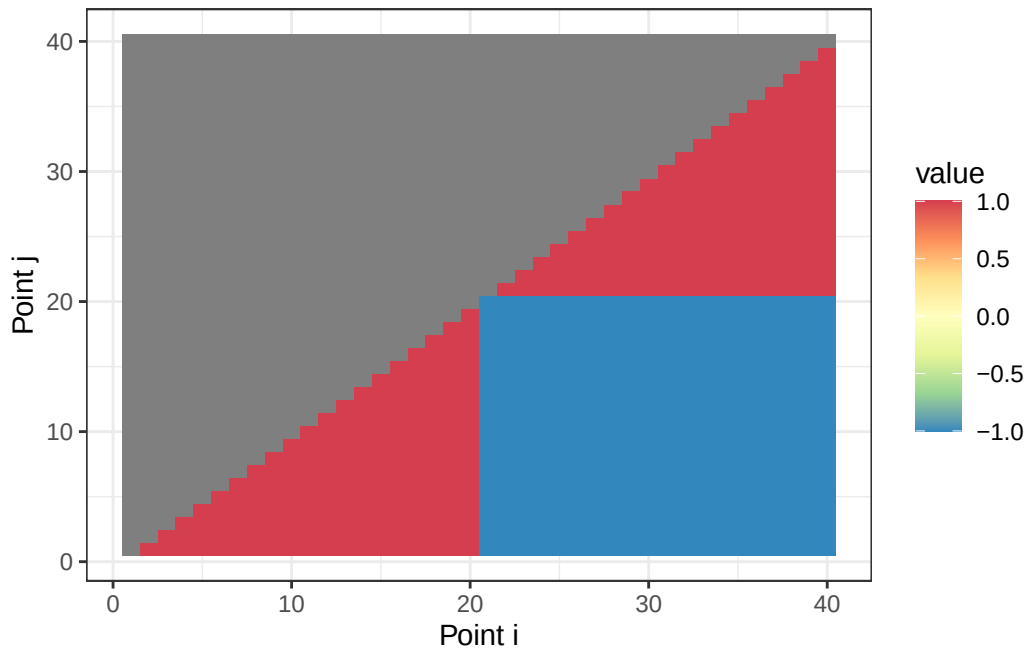

Figure 3.2: CRAB algorithm ran on Section 2.1 Dataset with k = 2 clusters

On the heatmap, values (i, j) denote the average (i, j) resample matrix values, excluding NA values where either points did not appear in the dataset. The gray values are removed because the matrix is symmetrical. Points that have values that are closer to '1' can be interpreted as being 'buddies', or in the same cluster, on average. Points that have values that are closer to '-1' can be interpreted as being 'enemies', or not in the same cluster, on average.

Because the algorithm was ran on a dataset where there was a clear distinction between the 2 clusters and the number of clusters inputted into the algorithm was set at 2, there are no situations where points go from buddies to enemies and vice versa. In this example, points, on average, are always consistently buddies or enemies across resamples.

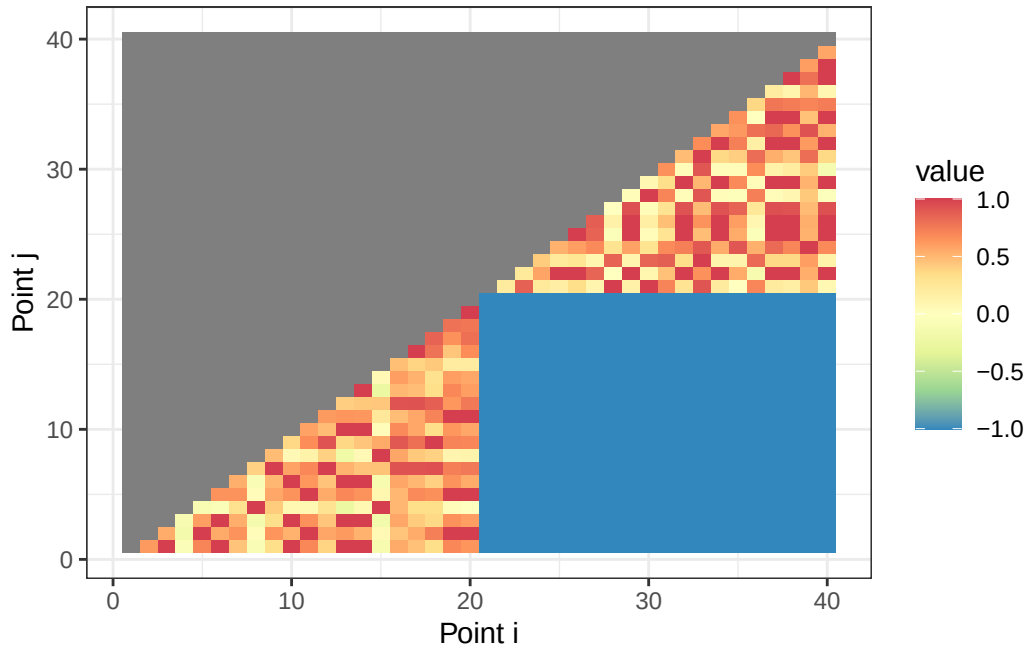

Figure 3.3: CRAB algorithm ran on Section 2.1 Dataset with k = 3 clusters

When running the CRAB algorithm with k = 3 clusters inputted, the composition of the mean matrix is significantly different compared to the previous example with k = 2. The values in the mean matrix that are close to '0' demonstrate point pairwise relationships that switch between being buddies and being enemies. Since k = 3 is clearly not a good fit for the sample dataset, the mean matrix reflects that through higher instability in the matrix.

3.3 Score Construction

To illustrate the construction of the final score given this mean matrix, we use the penguins dataset in R with three known species and scaled bill length and flipper lengths.

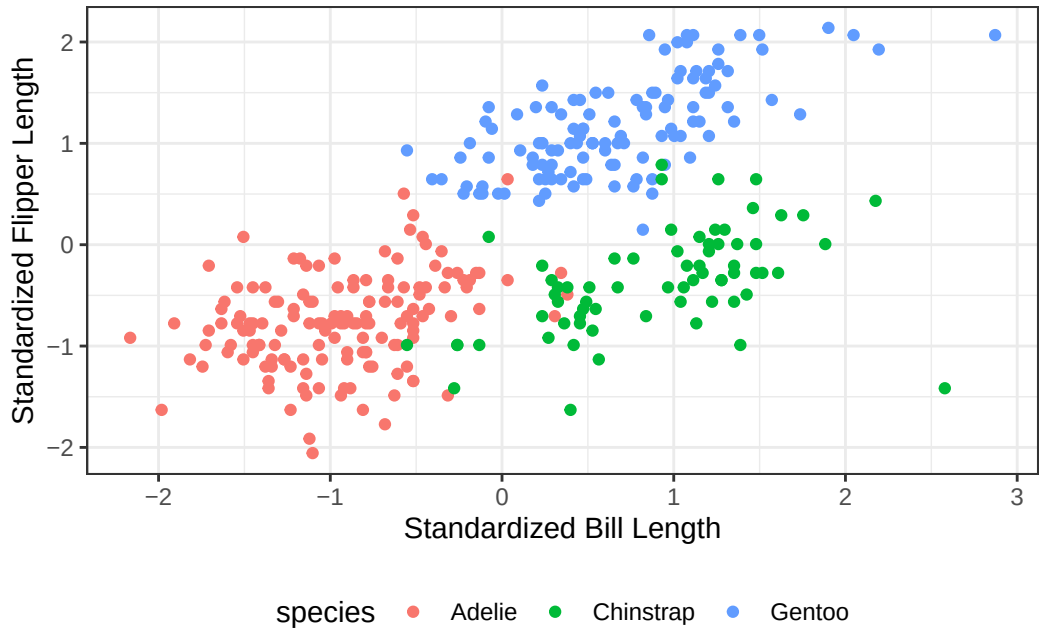

Figure 3.4: Plot of Penguin Species by Standardized Bill and Flipper Length

For the purposes of designing the scoring algorithm, we used this penguins dataset with the three "known" clusters in order to fine-tune the behavior of the scores. Ideally a scoring algorithm would output a higher score (worse) when the number of clusters input into the algorithm is not equal to three; the ideal output of the scoring algorithm when the number of clusters, k, is equal to 3 would be the minimum score over all k candidate cluster values.

Before defining this algorithm, it was helpful to first see the raw mean vector of values after averaging the p_{ij} values over all the m resample matrices. Note that the values of ones and negative ones were removed, since a vast majority of points either consistently are in the same cluster or are consistently not in the same cluster.

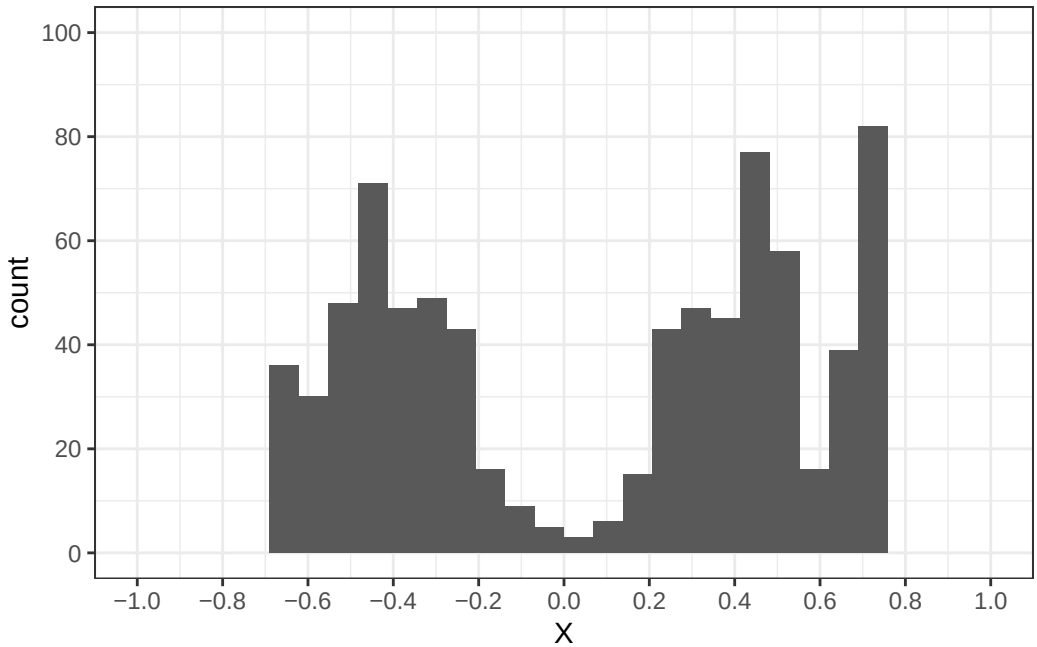

Figure 3.5: The distribution of values inside the mean matrix after running the algorithm on the penguins dataset, using K-Means and k = 2

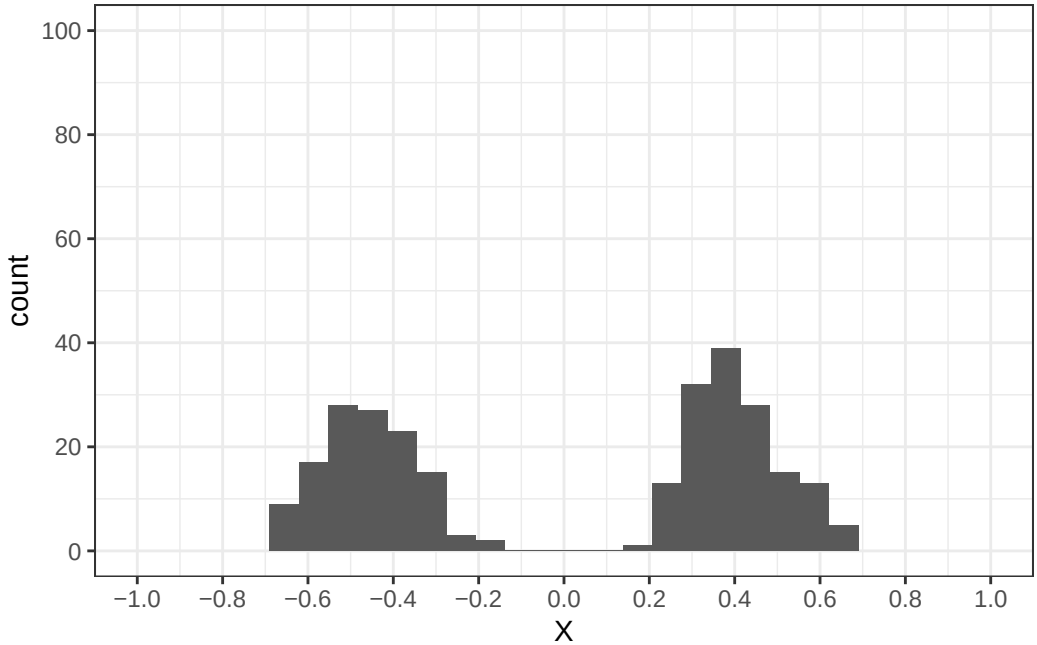

Figure 3.6: The distribution of values inside the mean matrix after running the algorithm on the penguins dataset, using K-Means and k = 3

We see that from Figure 3.2 and Figure 3.3 that the distribution of values inside the mean

matrix changes depending on the number of clusters set beforehand in the K-Means algorithm, albeit slightly. For k = 2, there are a small number of points around the value of 0, which indicates that these specific points are ones who are changing "allegiances" often, meaning that they switch from being "buddies" (being in the same cluster) and to being "rivals" (not being in the same cluster).

To incorporate all of this information, our algorithm heavily penalized points that constantly change allegiances and not penalize points that are consistent (whether consistently rivals or consistently buddies). Pairs of observations that are consistently in the same cluster would have an average p_{ij} value of 1 or close, and pairs of observations that are consistently not in the same cluster would have an average p_{ij} value of -1 or close. Then, pairs of observations that switch allegiances would have p_{ij} values close to 0.

For the final score, we took the absolute value of the vectorized mean matrix, which as a side effect gave consistent buddies and consistent rivals the same weight in our score. With this, we obtained the sum of squared distances from 1, allowing us to penalize those pairs of observations that were inconsistent.

Therefore, the CRAB Score is formally defined as:

$$\text{CRAB} = \frac{\sum_i^n \sum_j^n (1 - |\bar{p}_{ij}|)^2}{n^2}$$

where $\bar{p}_{ij}$ is the average of the p_{ij} values over all m resample matrices. We define the 'mean matrix' to be the $\bar{p}$ matrix. The 'mean matrix' function used in the algorithm is available in Appendix B, while the score calculation using this matrix is available in Appendix C.

The CRAB Score has a general tendency to increase depending on the sample size, n, which required us to take an average instead of the direct sum. This also allows us to compare CRAB Scores across datasets of different sizes since we now had a standardized score.

3.4 Speed and Memory Scaling

TODO: Explain Why important for speed and memory scaling.

3.4.1 Scaling with the Number of Observations

As mentioned in Section 3.1, the CRAB algorithm involves the construction of m resample matrices of size $n \times n$, where n is the number of observations in the dataset. Predictably, the time complexity of the algorithm roughly scales by the order of $m \cdot n^2$, without taking into account the complexity of the resample matrix construction or the clustering algorithm itself. This is because the m $n \times n$ resample matrices are reduced to a single mean matrix for the calculation of the CRAB score.

The k-Means algorithm has a time complexity that scales by the order of $n \cdot p \cdot q \cdot k$, where n is the number of observations, p is the number of dimensions, q is the number of iterations, and k is the number of clusters (Wang & Song, 2011).

Here we will focus on assessing the time complexity scale for the number of observations and the number of dimensions. The changes in the time compelxity of the k-Means algorithm itself, which is run for each resample in the CRAB algorithm, are negligible due to the small number and range of clusters.

Taking these time complexities into account, we can expect the entire algorithm to roughly scale at $O(m \cdot (n^2 + np))$, where m is the number of resample matrices, n is the number of observations, and p is the number of dimensions of the dataset.

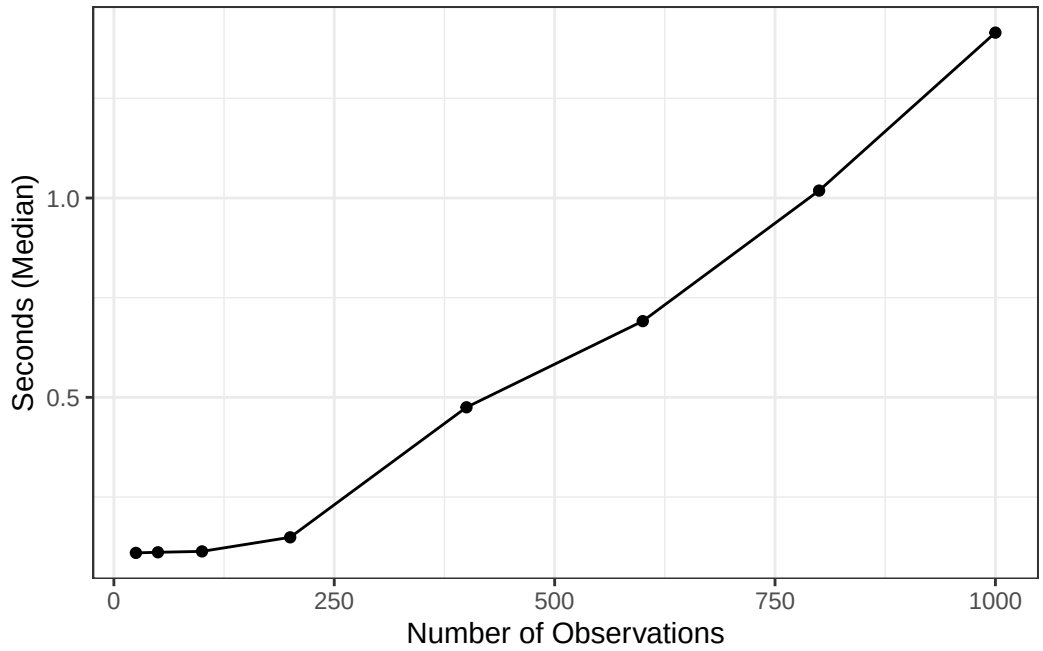

Figure 3.7: Speed of the CRAB Algorithm Quadratically Increases as Number of Observations Increases

As for memory scaling, due to the fact that m total $n \times n$ resample matrices are constructed, we can also expect memory usage to scale quadratically with respect to the number of observations, regardless of the clustering algorithm used. For this experiment, we hold constant the number of clusters k at 3 as well as hold the number of resamples to be 50, after which we vary the number of dimensions (p) and the number of observations (n); in actuality to compare the viability of each k the CRAB algorithm would need to be run k times, so once for each k of interest. The full details and code of the implementation are available on the GitHub repository under **analyses/speed_memory_scaling.qmd**.

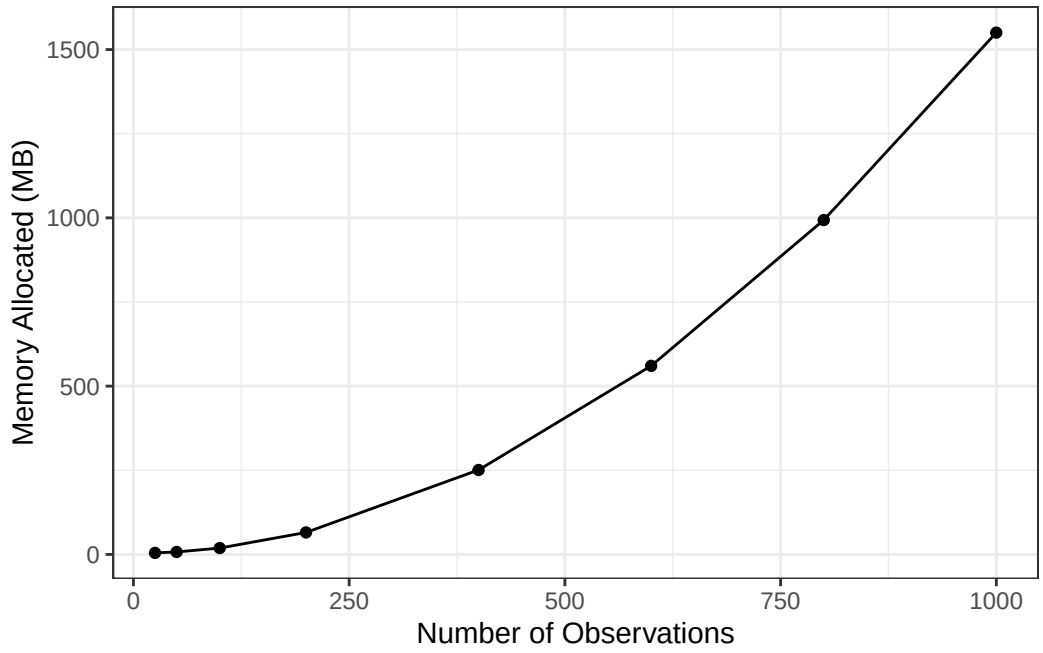

Figure 3.8: Memory of the CRAB Algorithm Quadratically Increases as Number of Observations Increases

TODO: Add table with n and p

This, of course, is not ideal, but does guide possible future work improving the algorithm. At a large n the resample matrices become extremely sparse, leading to possible memory savings if treated as such. However, the fact remains that the algorithm relies on the storage of all m resample matrices in some capacity.

3.4.2 Scaling with the Number of Dimensions

Unlike when changing the number of observations, increasing the number of dimensions should not have a large impact on the speed and memory performance of the CRAB algorithm itself. Since we rely on the clustering assignments output from the clustering algorithm for the CRAB implementation, the actual differences in time and memory usage come almost completely from the clustering algorithm used. As stated earlier, the time complexity of the k-Means algorithm scales linearly, scaling with np.

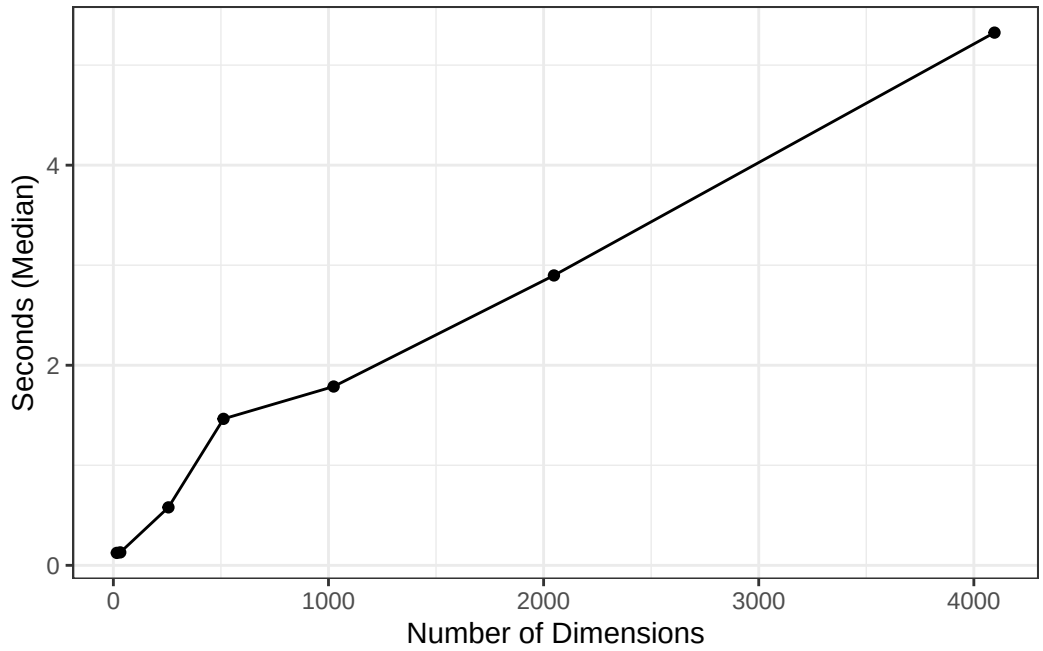

Figure 3.9: Speed of the CRAB Algorithm Increasing Linearly as Number of Dimensions Increases

This is verified through simulation where running the CRAB algorithm on increasing p dimensions only scales the time linearly with respect to p. The k-Means algorithm has a space complexity of $O((n + k) \cdot p)$, where p is the number of dimensions of the dataset (Ikotun et al., 2023). If we hold n and k to be constant, we would also expect the algorithm to scale linearly in terms of time in respect to the number of dimensions p.

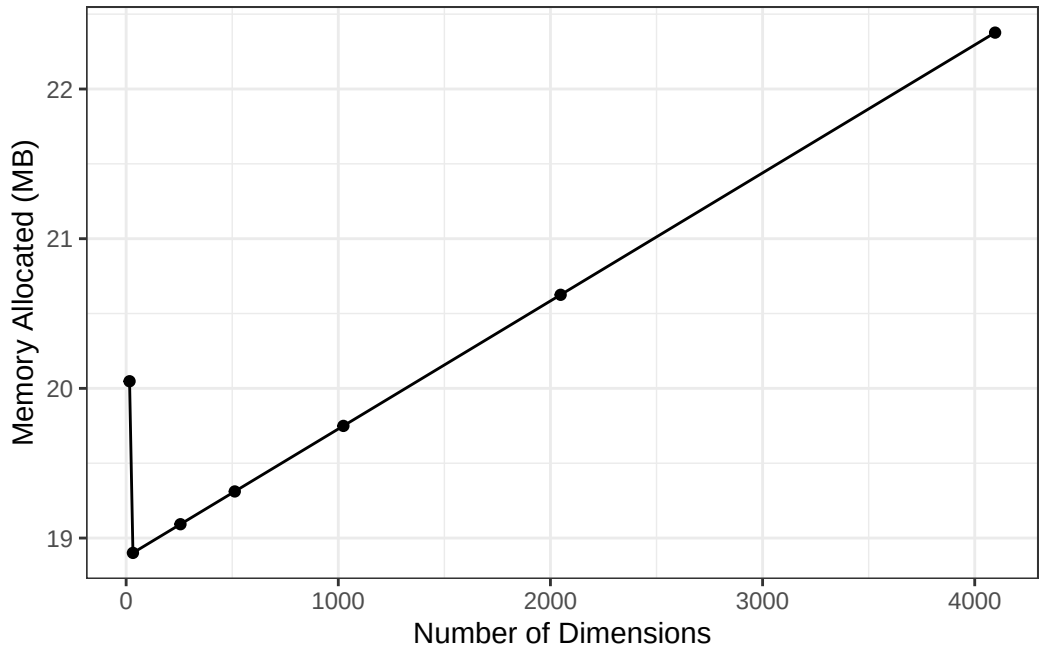

Figure 3.10: Memory of the CRAB Algorithm Increasing Linearly as Number of Dimensions Increases

Through simulation, barring the anomalous behavior with a small number of dimensions, the expected memory usage almost exactly follows this behavior where the memory allocation increases linearly with respect to p dimensions.

Chapter 4

NULL DISTRIBUTION

4.1 Null Distribution of Scores

TODO: Beef up

An important question in the construction of a score describing the optimal number of clusters is its behavior under a null condition: behavior when no clusters are present. Any anomalous behavior here where the scores trend in either an upward or downward direction is problematic; this suggests a bias in either direction when the algorithm is ran on data where clustering behavior does exist.

Our scores were adjusted depending on the behavior of the CRAB scores when there are in actuality no clusters (or equivalently if all points are in one large clsuter). Based off of this "null distribution", we adjusted our CRAB Scores to counteract the bias and downward trend in Scores.

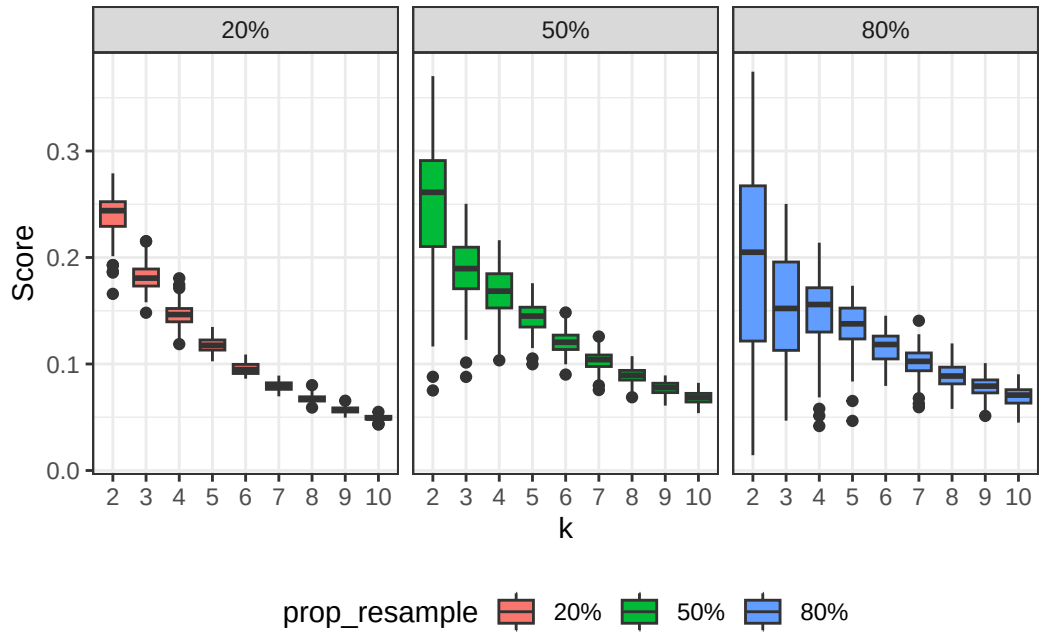

Figure 4.1: Distribution of Scores on a Random 2-Dimensional Normal Distribution, Faceted by Resample %

We wanted to see the exact effects of the various parameters in the algorithm and their effects on the CRAB algorithm. To see this specifically, we ran simulations on random normally distributed data where there were no visible clusters. There were 100 simulation runs per cluster, with 100 resample matrices and n = 100 per sample. Datasets across resampling percentages were the same.

We see in the above figure that the resample proportion has a significant impact on the distribution of scores, where the variability and mean of scores over many simulation runs is dependent on the resample proportion. Interestingly, the variability of the scores seemed to decrease overall on average when holding the number of clusters and sample size constant. This suggests that the resample percentage parameter has a significant impact on the end result of the algorithm, although more work needs to be done on how exactly this affects the results of the CRAB algorithm on datasets where clusters exist.

More details regarding the simulation can be found in **analyses/resampling_percent-**

age.qmd.

4.2 Null Distribution over Change in Number of Dimensions

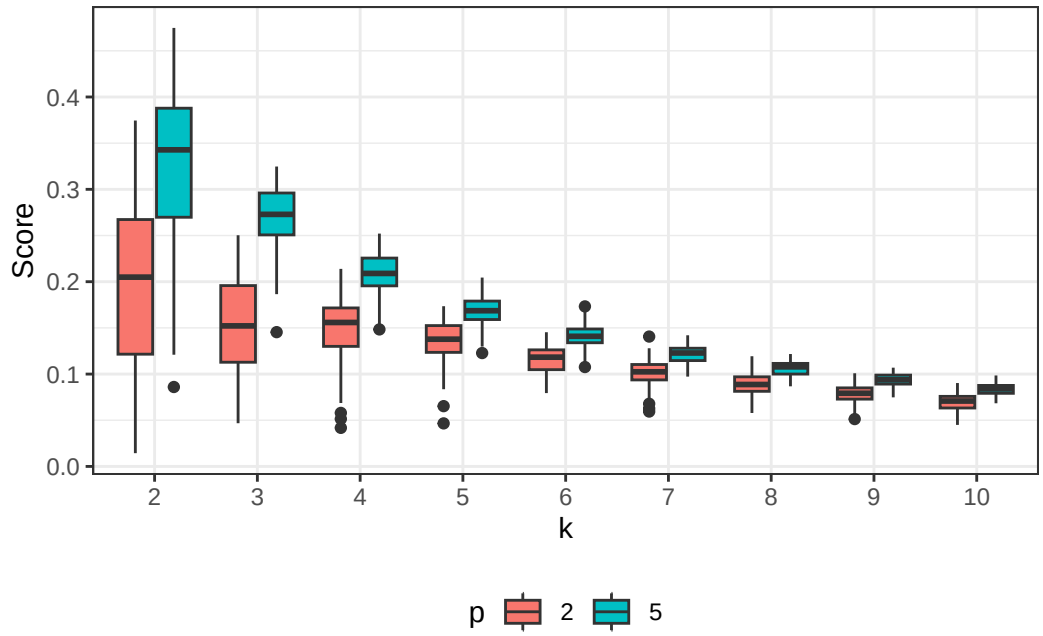

Figure 4.2: Distribution of Scores on a p-Dimensional Normal Distribution, Faceted by Number of Dimensions

When we examined the behavior of the CRAB algorithm as we scale the number of dimensions, we saw drastically different behavior in the distributions of CRAB Scores. Although it appears that as the number of clusters increases, the CRAB Scores in either case asymtotically level off, the behavior is significantly more right-skewed in the $p = 5$ case compared to the $p = 2$ case. Since we prefer a lower score when choosing the number of clusters, this suggested our algorithm biases for a higher number of clusters when the number of dimensions increases. The number of observations across these two datasets is kept constant at $n = 100$, which may also suggest the curse of dimensionality is contributing here as well.

4.3 Null Distribution over Change in Number of Observations

One of the most important properties to look into for the CRAB algorithm is how it performs under varying dataset sizes n. Since real-world datasets can range from as few as a couple observations to thousands or millions of observations, it is important to make sure the performance and behavior of the algorithm stays consistent with varying n. Of particular interest was this behavior on datasets with no clusters; differing null distributions suggest a bias in either direction as sample size changes.

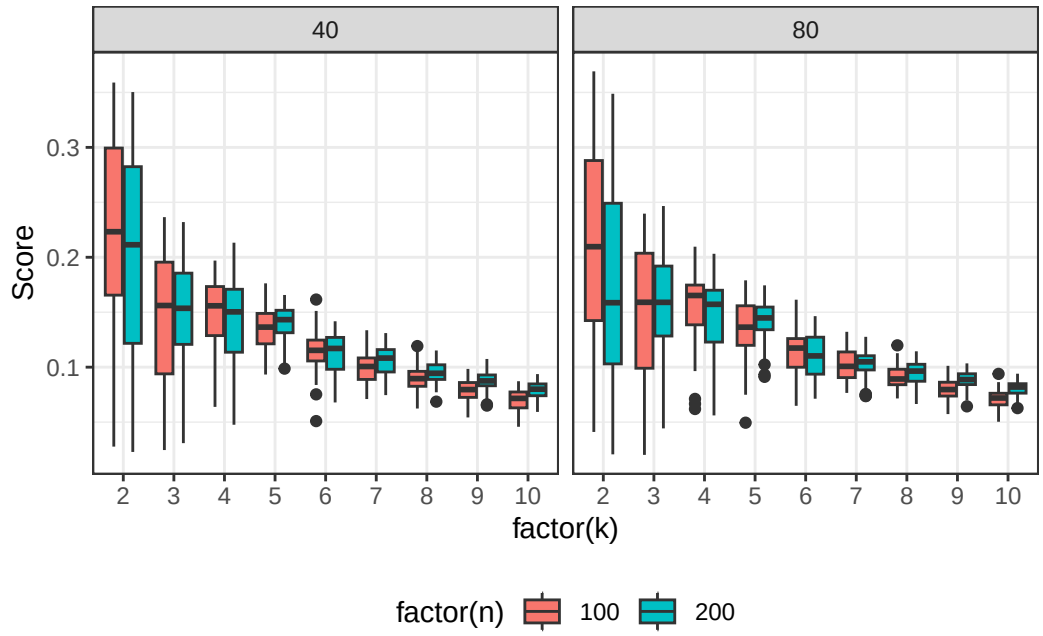

Figure 4.3: Distribution of scores on random p-dimensional normal distribution (proportion resample set to 80%)

Higher sample sizes seem to have asymptotically higher scores on average compared to lower sample sizes, though with smaller values of k the datasets with smaller sample sizes return higher values of the CRAB score on average. (TODO)

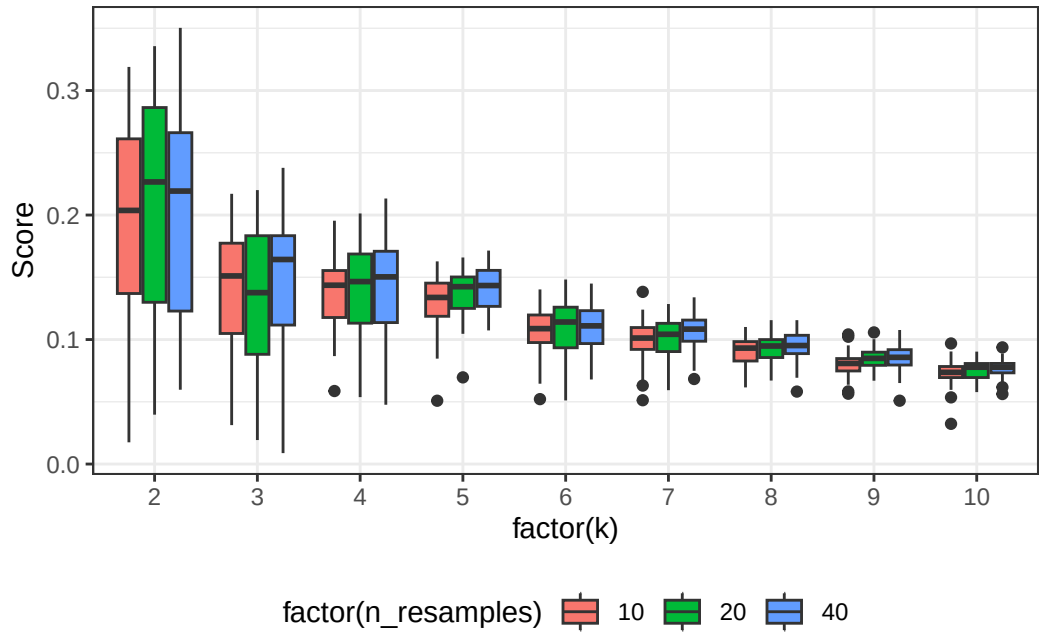

Figure 4.4: Distribution of scores across various resample percentages on random p-dimensional normal distribution (n = 200)

Not really any differences on the null distribution when the number of resamples varies. (TODO)

4.4 Adj-CRAB Score

The need to adjust the raw CRAB scores then becomes obvious; if the score through simulation displays monotonically decreasing behavior, then we should adjust any scores that we get as a function of the parameters of interest as necessary (n, resample proportion, k). To find a good approximation of this, we ran 50 simulated 2-dimensional normal distribution datasets, and for each dataset ran the CRAB algorithm from 2 to 30 clusters. The median CRAB score with respect to each cluster k was then taken.

```
         1          2          3          4          5          6          7
0.17327840 0.15736928 0.14282583 0.12956518 0.11750682 0.10657261 0.09668675
         8          9         10         11         12         13         14
```

```
0.08777585 0.07976884 0.07259704 0.06619413 0.06049615 0.05544151 0.05097099
        15         16         17         18         19         20         21
0.04702771 0.04355718 0.04050728 0.03782822 0.03547261 0.03339540 0.03155392
        22         23         24         25         26         27         28
0.02990787 0.02841929 0.02705261 0.02577460 0.02455443 0.02336359 0.02217598
        29
0.02096783
```

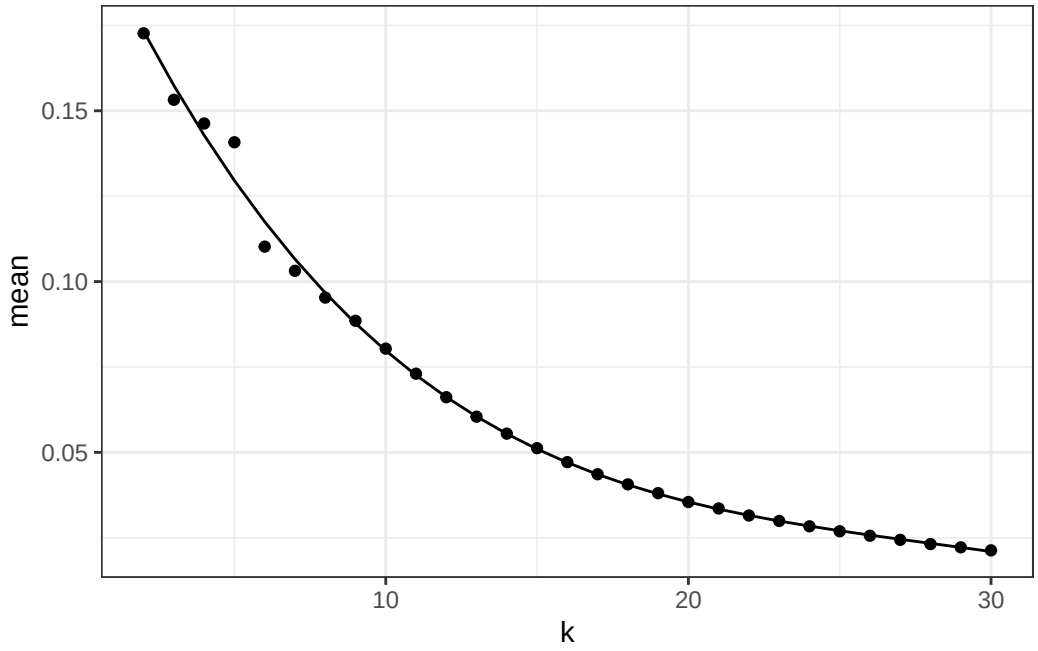

Figure 4.5: CRAB Score Behavior as k Increases (Fitted Curve vs. Actual Simulation Results) Approximated by a Quartic Regression

TODO

Chapter 5

CONCLUSION

TODO

5.1 Results

TODO

5.2 Limitations and Future Work

TODO

The current framework of the CRAB algorithm requires each observation to be assigned a cluster; note that this means that algorithms such as DBSCAN (Density-Based Spatial Clustering of Applications with Noise) will not work with this framework due to the fact that some observations are classified as “noise” (Ester, Kriegel, Sander, & Xu, 1996).

As explored in Section 3.4, the limitation of the CRAB algorithm in terms of computation time mainly stems from the loss in performance as the number of observations in the dataset increases.

No way to know if the number of clusters should just be 1 (no clusters) not really a problem since elbow method and silhouette score have the same constraint.

REFERENCES

Ababneh, J. I., Khodier, M. M., & Khatalin, S. M. (2025). Energy Optimization in Three-Dimensional Wireless Sensor Networks Using Hybrid K-Means and Mixed Integer Linear Programming. *IEEE Access*, *13*, 163276–163287. https://doi.org/10.1109/ACCESS.2025.3611305

Ben-Hur, A., Elisseeff, A., & Guyon, I. (2001). A stability based method for discovering structure in clustered data. *Biocomputing 2002*. Kauai, Hawaii, USA: WORLD SCIENTIFIC. https://doi.org/10.1142/9789812799623_0002

Bird, M. B., Mi, Q., Koltun, K. J., Lovalekar, M., Martin, B. J., Fain, A., … Nindl, B. C. (2022). Unsupervised Clustering Techniques Identify Movement Strategies in the Countermovement Jump Associated With Musculoskeletal Injury Risk During US Marine Corps Officer Candidates School. *Frontiers in Physiology*, *13*, 868002. https://doi.org/10.3389/fphys.2022.868002

Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226–231. Portland, Oregon: AAAI Press.

FISHER, R. A. (1936). THE USE OF MULTIPLE MEASUREMENTS IN TAXONOMIC PROBLEMS. *Annals of Eugenics*, *7*(2), 179–188. https://doi.org/https://doi.org/10.1111/j.1469-1809.1936.tb02137.x

Fränti, Pasi, & Sieranoja, S. (2018). K-means properties on six clustering benchmark datasets. Retrieved from http://cs.uef.fi/sipu/datasets/

Fränti, P., & Virmajoki, O. (2006). Iterative shrinking method for clustering problems. *Pattern Recognition*, *39*(5), 761–765. https://doi.org/10.1016/j.patcog.2005.09.012

Horst, A. M., Hill, A. P., & Gorman, K. B. (2022). Palmer archipelago penguins data in

the palmerpenguins r package - an alternative to anderson's irises. *The R Journal*, *14*, 244–254. https://doi.org/10.32614/RJ-2022-020

Ikotun, A. M., Ezugwu, A. E., Abualigah, L., Abuhaija, B., & Heming, J. (2023). K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data. *Information Sciences*, *622*, 178–210. https://doi.org/10.1016/j.ins.2022.11.139

Jaeger, A., & Banks, D. (2023). Cluster analysis: A modern statistical review. *WIREs Computational Statistics*, *15*(3). https://doi.org/10.1002/wics.1597

John, J. M., Shobayo, O., & Ogunleye, B. (2023). An Exploration of Clustering Algorithms for Customer Segmentation in the UK Retail Market. *Analytics*, *2*(4), 809–823. https://doi.org/10.3390/analytics2040042

Kodinariya, T., & Makwana, P. (2013). Review on Determining of Cluster in K-means Clustering. *International Journal of Advance Research in Computer Science and Management Studies*, *1*, 90–95.

Levine, E., & Domany, E. (2001). Resampling Method for Unsupervised Estimation of Cluster Validity. *Neural Computation*, *13*(11), 2573–2593. https://doi.org/10.1162/089976601753196030

MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, Volume 1: Statistics* (Vol. 5.1, pp. 281–298). University of California Press. Retrieved from https://projecteuclid.org/ebooks/berkeley-symposium-on-mathematical-statistics-and-probability/Proceedings-of-the-Fifth-Berkeley-Symposium-on-Mathematical-Statistics-and/chapter/Some-methods-for-classification-and-analysis-of-multivariate-observations/bsmsp/1200512992

Nugent, R., & Meila, M. (2010). An overview of clustering applied to molecular biology. *Methods in Molecular Biology*, *620*, 369–404. https://doi.org/10.1007/978-1-60761-580-4_12

Roper, L., He, V. Y., Perez-Concha, O., & Guthridge, S. (2023). Complex early childhood experiences: Characteristics of Northern Territory children across health, education and child protection data. *PloS One*, *18*(1), e0280648. https://doi.org/10.1371/journal.pone.0280648

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, *20*, 53–65. https://doi.org/https://doi.org/10.1016/0377-0427(87)90125-7

Schubert, E. (2023). Stop using the elbow criterion for k-means and how to choose the number of clusters instead. *ACM SIGKDD Explorations Newsletter*, *25*(1), 36–42. https://doi.org/10.1145/3606274.3606278

Thorndike, R. L. (1953). Who belongs in the family? *Psychometrika*, *18*(4), 267–276. https://doi.org/10.1007/BF02289263

Wang, H., & Song, M. (2011). Ckmeans.1d.dp: Optimal k-means Clustering in One Dimension by Dynamic Programming. *The R Journal*, *3*(2), 29–33. Retrieved from https://pmc.ncbi.nlm.nih.gov/articles/PMC5148156/

APPENDICES

Appendix A

RESAMPLE MATRIX

```
resample_function <- function(data = data,
                              formula = ~ .,
                              k = 3,
                              number_of_resamples = 15,
                              proportion_resample = 0.9,
                              starting_seed = 599,
                              algorithm = "kmeans") {
  data <- data |>
    drop_na()
  data$index <- 1:nrow(data)
  results <- list()

  # For loop
  results <- foreach(i = 1:number_of_resamples,
                     .packages = c("tidyclust",
                                   "dplyr",
                                   "tidymodels")) %dopar% {
    # Reproducibility over parallel
    set.seed(starting_seed + i)
    result_matrix <- matrix(0, nrow = nrow(data),
```

```
                          ncol = nrow(data))


random_sample <- data |>
  filter(index %in% sample(index,
                           proportion_resample * nrow(data)))


if (algorithm == "kmeans") {
  cluster_assigned <- k_means(num_clusters = k) |>
    fit({{formula}},
        data = random_sample |>
          select(-index))
} else if (algorithm == "hier_clust") {
  cluster_assigned <- hier_clust(num_clusters = k) |>
    fit({{formula}},
        data = random_sample |>
          select(-index))
} else {
  stop("Algorithm is not supported. Please select
       one of (kmeans, hier_clust)")
}



intermediate <- data.frame(random_sample$index,
                           extract_cluster_assignment(
                             cluster_assigned
                             ) |>
                             mutate(.cluster = as.character(
```

```
                                    .cluster)
                                    ),
                                stringsAsFactors = FALSE)
    colnames(intermediate) <- c("index", "cluster")

    for (c in unique(intermediate$cluster)) {
      idx <- intermediate[intermediate$cluster == c, ]$index

      # Check that the index list is not length 1 for
      # purposes of n > m for combn
      if (length(idx) > 1) {
        idx <- sort(idx)
        ones <- t(combn(idx, 2))
        result_matrix[cbind(ones[, 1], ones[, 2])] <- 1
      }

      neg_one_idx <- expand.grid(idx,
                                 setdiff(random_sample$index, idx))
      result_matrix[cbind(neg_one_idx[, 1], neg_one_idx[, 2])] <- -1
    }
    result_matrix[lower.tri(result_matrix, diag = TRUE)] <- NA
    result_matrix
   }
  return(results)
}
```

Appendix B

MEAN MATRIX

```r
mean_matrix <- function(list_of_matrices = list_of_matrices) {
  # Absolute value the entire matrix, for all matrices
  absolute_sum <- lapply(list_of_matrices, abs)

  # Get the sum of absolute values in the final matrix
  absolute_final <- absolute_sum |>
    reduce(`+`)

  # Get the sum of values in the final matrix
  numerator <- list_of_matrices |>
    reduce(`+`)

  # Returns the matrix of zero-removed means.
  return(numerator / absolute_final)
}
```

Appendix C

METRIC CALCULATION

```r
squared_distance_from_one <- function(mean_matrix = mean_matrix) {
  # Ignore the order of the elements in matrix
  res_vec <- as.vector(mean_matrix)

  # Absolute value and remove NAs
  res_vec <- abs(res_vec[!is.na(res_vec)])

  # Take average squared distance from one
  return(mean((1 - res_vec)^2))
}
```

Appendix D

POINT CONTRIBUTION FUNCTION

```
point_contribution <- function(data, mean_matrix) {
  # Returns each observation's contribution to the CRAB Score.
  # This is returned as an (index, score) dataframe.

  # Get a list of lists containing index and index "CRAB" score
  scores <- lapply(c(1:nrow(data)), function(i) {
    full_vec <- getPointVecHelper(i, mean_matrix)
    singlePointScore <- mean((1 - full_vec)^2)
    return(list(index = i, singlePointScore = singlePointScore))
  })
  final_df <- do.call(rbind.data.frame, scores)
  total_score <- mean(final_df$singlePointScore)

  final_df <- final_df |>
    mutate(singlePointContribution = singlePointScore / total_score)

  return(final_df)
}
```

Appendix E

GET POINT VECTOR HELPER

```
getPointVecHelper <- function(i, mean_matrix) {
  # Return vector of all values in the
  # mean matrix corresponding to obs i
  # after removing all NAs and then taking absolute value

  row <- mean_matrix[i, ]
  col <- mean_matrix[, i]

  # We don't care about the order, just the values themselves
  res_vec <- c(row, col)
  res_vec <- abs(res_vec[!is.na(res_vec)])
  return(res_vec)

}
```

Appendix F

CRAB SCORE ACROSS K CLUSTERS

```
crab_k_across <- function(data = data,
                          c_vec = c_vec,
                          formula = ~ .,
                          number_of_resamples = 80,
                          proportion_resample = 0.8,
                          starting_seed = 599,
                          algorithm = "kmeans") {
  # Calls the resampling function across the dataset,
  # with the parameters passed to resampling function.

  # Returns a dataframe with k rows
  # (one for each element in the c_vec cluster vector)
  # and a value for the cRab score after running the CRAB
  # algorithm on the dataset.

  result <- lapply(c(c_vec), function(k) {
    resample_matrices <- resample_function(
      data = data,
      formula = formula,
      k = k,
      number_of_resamples = number_of_resamples,
      proportion_resample = proportion_resample,
```

```
      starting_seed = starting_seed,
      algorithm = algorithm
      )

    m_matrix <- mean_matrix(resample_matrices)
    score <- squared_distance_from_one(m_matrix)

    return(list(k = k,
                CRAB = score))
  })
  return(dplyr::bind_rows(result))
}
```